תרגיל 6: תכנות מונחה עצמים

תאריך פרסום: 14.2.2024

תאריך הגשה: 3.3.2024 בשעה 23:59

מתרגלת אחראית: תום משיח

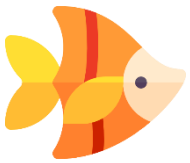
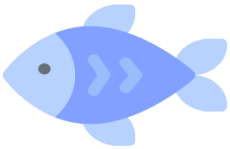
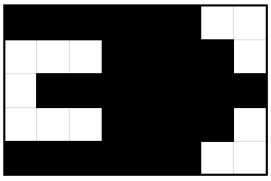
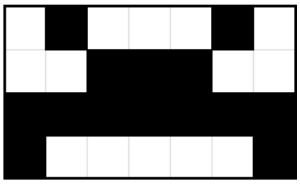
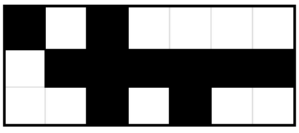
משקל תרגיל: 5 נקודות

דגשים:

1. מומלץ לקרוא את כל התרגיל **לפני** המימוש. אנו ממליצים להבין היטב את ארכיטקטורת המערכת ולתכנן אותה על דף נייר (או אייפד) לפני תחילת כתיבת הקוד.
2. בניגוד לתרגילים קודמים, **בתרגיל זה אין להניח שהקלט תקין!** עבור כל שיטה / פונקציה מוגדרות הדרישות לגבי הקלט. הנחות על קלטים מסוימים יצוינו באופן מפורש. ובמידה והקלט לא תקין עליכם לזרוק חריגה מתאימה.
3. מכיוון שהעבודה עמוסה מבחינת כמות הקוד שעליכם לממש וכדי להקל עליכם - אינכם נדרשים לכתוב docstring בעבודה זו.
4. בתרגיל **לא מצוין** באופן מפורש איך עליכם לממש את הפתרון. לדוגמא, לא יצוין מתי עליכם לדרוס שיטה, להשתמש בשיטת האב, להגדיר מחלקה\שיטה אבסטרקטית, להגדיר שדה פרטי, לבצע shallow copy או deep copy וכדומה. עליכם יהיה להחליט על פרטי המימוש **תוך עמידה בדרישות התרגיל ובעקרונות שלמדתם בקורס** (לדוגמא, להמנע משכפול קוד).
5. נא לקרוא את כל העבודה **לפני** שאתם מתחילים לכתוב את הקוד (זו הפעם השניה של הסעיף הזה ולא במקרה..).

האקווריום

במטלה זו תממשו אקווריום שיהיה מיוצג באמצעות רשימה מקוננת (מטריצה). כל תא באקווריום יכול להכיל מופע של בעל חיים (**Animal**): אחד משני סוגים של דג (**Fish**) ואחד משני סוגים של סרטן (**Crab**), שמתוארים באיור (השורה התחתונה מכילה ייצוג ויזואלי של בעל החיים).

Animal			
Fish		**Crab**	
Scalar	**Molly**	**Ocypode**	**Shrimp**
			
5X8 	3X8	4X7 	3X7 

החיים באקווריום מתנהלים בסבבים, כך שכל סבב מייצג יחידת זמן. בכל סבב, כל מופע של בעל חיים יוכל לנוע לתא סמוך, לאכול, ולהזדקן.

עליכם יהיה לממש את האקווריום לפי עקרונות תכנות מונחה עצמים (OOP), בפרט מודולריות - modularity, אבסטרקציה – abstraction, והכמסה - encapsulation.

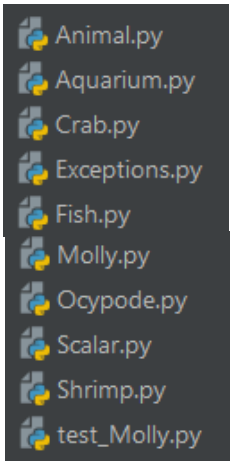
ב-GIF שבקובץ "OOP GIF" ניתן לראות דוגמא לסבבים 0-25 באקווריום הכוללים תנועה של בעלי חיים, הזדקנות, ושינוי בכמות האוכל הזמינה להם. שימו לב למופע של בעל חיים שנפטר בשיבה טובה.

מבנה ההגשה:

<u>שימו לב:</u> עליכם להגיש בעבודה זו 10 קבצים. שמות הקבצים צריכים להיות תואמים בדיוק להנחיות. עליכם לשמור ב-PyCharm את כל הקבצים באותה תיקייה על מנת שתוכלו לייבא (import) אותם באמצעות שמם (ללא נתיב לקובץ).

לדוגמא: `from Fish import Fish`

לפניכם רשימת הקבצים שיש להגיש (שימו לב, אין צורך להגיש את קובץ ה-main).



חלק א':

בחלק זה יתוארו מחלקות החריגות (Exceptions), וכן המחלקות הבאות: Scalar, Molly, Ocypode, Shrimp ו-Aquarium, אשר תממשו בתרגיל. בנוסף, עליכם לממש את המחלקות Animal, Fish ו-Crab על פי עקרונות תכנות מונחה עצמים. כל מחלקה נמצאת בקובץ נפרד כך ששם הקובץ הוא שם המחלקה.

חריגות (Exceptions)

בקובץ בשם Exceptions.py, עליכם לממש מספר מחלקות של חריגות שישמשו אתכם בהמשך התרגיל (יהיה עליכם לזרוק את החריגות המתאימות בהתאם לדרישות שיפורטו ביתר המחלקות).

InvalidInputException

החריגה תיזרק כאשר הקלט לא תואם את הדרישות.

NotAvailablePlace

החריגה תיזרק כאשר מנסים להכניס לאקווריום חיה במקום שתפוס על ידי חיה אחרת.

TooSmallAquariumSize

החריגה תיזרק כאשר מנסים לבנות אקווריום בגודל קטן מהגודל המינימלי המותר.

InvalidAnimalType

החריגה תיזרק כאשר מנסים להכניס לאקווריום חיה שלא קיימת.

בחריגה זו בלבד עליכם לדרוס את השיטה __str__ של המחלקה InvalidAnimalType:

def __str__(self):

השיטה תחזיר מחרוזת המייצגת תיאור של החריגה.

<u>פלט:</u>

- [str]- מחרוזת המייצגת את השגיאה באופן הבא:

Error: "{animal}" is an invalid animal type. The valid animal types are: molly, scalar, ocypode, shrimp.

דוגמא:

```
try:
  raise InvalidAnimalType ("cat")
except Exception as e:
  print(e)
```

יודפס:

'Error: "cat" is an invalid animal type. The valid animal types are: molly, scalar, ocypode, shrimp.'

Scalar:

מחלקה אשר מייצגת דג מטיפוס Scalar (לא הביטוי המתמטי!) המיוצג על ידי השדות הבאים:

- **name**- משתנה מטיפוס מחרוזת לא ריקה המייצגת את שם החיה.
- **age**- משתנה מטיפוס int, מספר שלם הגדול מ-0 וקטן מ-120 המייצג את גיל החיה.
- **food**- משתנה מטיפוס int, המייצג את כמות האוכל הזמינה לחיה. כאשר נוצר מופע חדש של Scalar, ערכו של המשתנה food מאותחל ב-10.
- **width**- משתנה מטיפוס int המייצג את רוחב החיה. רוחב ה-Scalar מאותחל ב-8.
- **height**- משתנה מטיפוס int המייצג את גובה החיה. גובה ה-Scalar מאותחל ב-5.
- **directionH**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האופקי של ה-Scalar באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- שמאלה, 1- ימינה.
- **directionV**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האנכי של ה-Scalar באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- למטה, 1- למעלה.
- **x**- משתנה מטיפוס int המייצג את מיקום ה-Scalar בציר ה-x באקווריום, מספר שלם הגדול מ-0.
- **y**- משתנה מטיפוס int המייצג את מיקום ה-Scalar בציר ה-y באקווריום, מספר שלם הגדול מ-0.

<u>ממשו את מחלקת Scalar:</u>

def __init__(self, name, age, x, y, directionH, directionV):

בנאי המאתחל את שדות המחלקה.

במידה ואחד השדות קיבל ערך לא תקין יש לזרוק חריגה מתאימה.

def __str__(self):

השיטה תחזיר מחרוזת המייצגת דג Scalar בפורמט:

The scalar name is age years old and has food food.

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

לדוגמא:

The scalar myscalar is 12 years old and has 10 food.

def get_animal(self):

השיטה מחזירה רשימה מקוננת המכילה את צורת ה-Scalar בכיוון ההתקדמות שלו. שימו לב כי צורת ה-Scalar נקבעת לפי כיוון ההתקדמות בציר האופקי בלבד (ימינה/שמאלה).

לדוגמא עבור דג Scalar המתקדם בכיוון שמאל תוחזר הרשימה הבאה (הרשימות נתונות עבורכם בקובץ animals_lists.py):

```
[[" ", " ", "*", "*", "*", "*", "*", "*"],
 [" ", "*", "*", "*", " ", " ", " ", " "],
 ["*", "*", "*", "*", "*", "*", " ", " "],
 [" ", "*", "*", "*", " ", " ", " ", " "],
 [" ", " ", "*", "*", "*", "*", "*", "*"]]
```

def __repr__ (self):

השיטה מחזירה מחרוזת על פי הרשימה המקוננת המתקבלת מהשיטה get_animal. חיבור מחרוזת הפלט יתבצע ע"י חיבור איברי הרשימה הפנימית מופרדים אחד מהשני בתו ריק. בנוסף, בסוף כל רשימה פנימית יוסף למחרוזת '\n' כלומר, שורה חדשה.

לדוגמא עבור דג Scalar המתקדם בכיוון שמאל תוחזר המחרוזת הבאה:

```
      * * * * * *
    * * *
  * * * * * *
    * * *
      * * * * * *
```

def get_position (self):

השיטה מחזירה טאפל (tuple) המייצג את מיקום דג ה-Scalar באופן הבא: (x,y).

def get_directionH (self):

השיטה מחזירה את כיוון ההתקדמות בציר האופקי של דג ה-Scalar.

def get_directionV (self):

השיטה מחזירה את כיוון ההתקדמות בציר האנכי של דג ה-Scalar.

def get_size (self):

השיטה מחזירה טאפל (tuple) המייצג את גודל דג ה-Scalar באופן הבא: (width,height).

def dec_food (self):

השיטה מעדכנת את כמות האוכל הזמינה לדג ה-Scalar ומחסירה ממנו מנה אחת.

def add_food(self, amount):

השיטה מעדכנת את כמות האוכל הזמינה לדג ה-Scalar ומעלה אותו ב-amount כך ש-amount הינו משתנה מטיפוס int הגדול מ-0. (ניתן להניח כי הקלט תקין).

def inc_age (self):

השיטה מעדכנת את גיל דג ה-Scalar ומעלה אותו ב-1.

def move (self):

השיטה מזיזה את דג ה-Scalar צעד אחד לפי כיוון ההתקדמות שלו. בכל צעד, דג ה-Scalar מתקדם יחידה אחת בציר האופקי ויחידה אחד בציר האנכי, כלומר נע באלכסון. כיוון ההתקדמות בכל ציר נקבע על פי הערכים בשדות directionH ו-directionV והצעד מתבצע על ידי שינוי הערכים בשדות x ו-y בנדרש.

התדקמות שמאלה = הקטנת הערך בציר ה-x ביחידה.

התקדמות ימינה = הגדלת הערך בציר ה-x ביחידה.

התדקמות למטה = הגדלת הערך בציר ה-y ביחידה.

התקדמות למעלה = הקטנת הערך בציר ה-y ביחידה.

* שימו לב שבשלב זה אין צורך לדאוג לכך שהערכים ב-x ו-y עלולים להיות שליליים.

דומאת הרצה:

```
my_scalar = Scalar("myScalar", 12, x=10, y=5, directionH=1, directionV=1)
print(f'The position of the scalar is: x={my_scalar.get_position()[0]} , y={my_scalar.get_position()[1]}')
my_scalar.move()
print(f'The new position of the scalar is: x={my_scalar.get_position()[0]} , y={my_scalar.get_position()[1]}')
```

```
The position of the scalar is: x=10 , y=5
The new position of the scalar is: x=11 , y=4
```

def set_directionH(self, directionH):

השיטה מעדכנת את כיוון ההתקדמות של דג ה-Scalar בהתאם למשתנה הקלט directionH (ניתן להניח ש-directionH הינו מטיפוס int השווה ל-0 או 1).

def set_directionV(self, directionV):

השיטה מעדכנת את כיוון ההתקדמות של דג ה-Scalar בהתאם למשתנה הקלט directionV (ניתן להניח ש-directionV הינו מטיפוס int השווה ל-0 או 1)

def starvation (self):

השיטה בודקת האם דג ה-Scalar גווע מרעב על ידי בדיקת כמות האוכל הזמינה שנותרה לו. במידה וכמות האוכל של דג ה-Scalar הינה 0 השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died at the age of age years because it ran out of food.

אחרת, תחזיר False.

def die (self):

השיטה בודקת האם דג ה-Scalar מת בשיבה טובה על ידי בדיקת גילו. במידה וגילו הינו 120 ומעלה, השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died in a good health.

אחרת, תחזיר False.

Molly:

מחלקה אשר מייצגת דג מטיפוס Molly המיוצג על ידי השדות הבאים:

- **name**- משתנה מטיפוס מחרוזת לא ריקה המייצגת את שם החיה.
- **age**- משתנה מטיפוס int, מספר שלם הגדול מ-0 וקטן מ-120 המייצג את גיל החיה.
- **food**- משתנה מטיפוס int, המייצג את כמות האוכל הזמינה לחיה. כאשר נוצר מופע חדש של Molly, ערכו של המשתנה food מאותחל ב-10.
- **width**- משתנה מטיפוס int המייצג את רוחב החיה. רוחב ה-Molly מאותחל ב-8.
- **height**- משתנה מטיפוס int המייצג את גובה החיה. גובה ה-Molly מאותחל ב-3.
- **directionH**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האופקי של ה-Molly באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- שמאלה, 1- ימינה.

- **directionV**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האנכי של ה-Molly באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- למטה, 1- למעלה.
- **x**- משתנה מטיפוס int המייצג את מיקום ה-Molly בציר ה-x באקווריום, מספר שלם הגדול מ-0.
- **y**- משתנה מטיפוס int המייצג את מיקום ה-Molly בציר ה-y באקווריום, מספר שלם הגדול מ-0.

<u>ממשו את מחלקת Molly:</u>

def __init__(self, name, age, x, y, directionH, directionV):

בנאי המאתחל את שדות המחלקה.

במידה ואחד השדות קיבל ערך לא תקין יש לזרוק חריגה מתאימה.

def __str__(self):

על השיטה להחזיר מחרוזת המייצגת דג ה-Molly בפורמט:

The molly name is age years old and has food food.

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

לדוגמא:

The molly mymolly is 13 years old and has 10 food.

def get_animal(self):

השיטה מחזירה רשימה מקוננת המכילה את צורת דג ה-Molly בכיוון ההתקדמות שלו. שימו לב כי צורת דג ה-Molly נקבעת לפי כיוון ההתקדמות בציר האופקי בלבד (ימינה/שמאלה).

לדוגמא עבור דגMolly המתקדם בכיוון ימין תוחזר הרשימה הבאה:

```
[["*", " ", " ", "*", "*", "*", "*", " "],
["*", "*", "*", "*", "*", "*", "*", "*"],
["*", " ", " ", "*", "*", "*", "*", " "]]
```

def __repr__ (self):

השיטה מחזירה מחרוזת על פי הרשימה המקוננת המתקבלת מהשיטה get_animal. חיבור מחרוזת הפלט יתבצע ע"י חיבור איברי הרשימה הפנימית מופרדים אחד מהשני בתו ריק. בנוסף, בסוף כל רשימה פנימית יוסף למחרוזת '\n' כלומר, שורה חדשה.

לדוגמא עבור דג ה-Molly המתקדם בכיוון שמאל תוחזר המחרוזת הבאה:

def get_position (self):

השיטה מחזירה טאפל (tuple) המייצג את מיקום דג ה-Molly באופן הבא: (x,y).

def get_directionH (self):

השיטה מחזירה את כיוון ההתקדמות של דג ה-Molly בציר האופקי.

def get_directionV (self):

השיטה מחזירה את כיוון ההתקדמות של דג ה-Molly בציר האנכי.

def get_size (self):

השיטה מחזירה טאפל (tuple) המייצג את גודל ה-molly באופן הבא: (width,height).

def dec_food (self):

השיטה מעדכנת את כמות האוכל הזמינה לדג ה-Molly ומחסירה ממנו מנה אחת.

def add_food(self, amount):

השיטה מעדכנת את כמות האוכל הזמינה לדג ה-Molly ומעלה אותו ב-amount כך ש-amount הינו משתנה מטיפוס int הגדול מ-0. (ניתן להניח כי הקלט תקין).

def inc_age (self):

השיטה מעדכנת את הגיל של ה-Molly ומעלה אותו ב-1.

def move (self):

השיטה מזיזה את דג ה-Molly צעד אחד לפי כיוון ההתקדמות שלו. בכל צעד, דג ה-Molly מתקדם יחידה אחת בציר האופקי ויחידה אחד בציר האנכי, כלומר נע באלכסון. כיוון ההתקדמות בכל ציר נקבע על פי הערכים בשדות directionH ו-directionV והצעד מתבצע על ידי שינוי הערכים בשדות x ו-y כנדרש.

התדקמות שמאלה = הקטנת הערך בציר ה-x ביחידה.

התקדמות ימינה = הגדלת הערך בציר ה-x ביחידה.

התדקמות למטה = הגדלת הערך בציר ה-y ביחידה.

התקדמות למעלה = הקטנת הערך בציר ה-y ביחידה.

* שימו לב שבשלב זה אין צורך לדאוג לכך שהערכים ב-x ו-y עלולים להיות שליליים.

def set_directionH(self, directionH):

השיטה מעדכנת את כיוון ההתקדמות של דג ה-Molly בהתאם למשתנה הקלט directionH (ניתן להניח ש-directionH הינו מטיפוס int השווה ל-0 או 1).

def set_directionV(self, directionV):

השיטה מעדכנת את כיוון ההתקדמות של דג ה-Molly בהתאם למשתנה הקלט directionV (ניתן להניח ש-directionV הינו מטיפוס int השווה ל-0 או 1)

def starvation (self):

השיטה בודקת האם דג ה-Molly גווע מרעב על ידי בדיקת כמות האוכל הזמינה שנותרה לו. במידה וכמות האוכל של דג ה-Molly הינה 0 השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died at the age of age years because it ran out of food.

אחרת, תחזיר False.

def die (self):

השיטה בודקת האם דג ה-Molly מת בשיבה טובה על ידי בדיקת גילו. במידה וגילו הינו 120 ומעלה, השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died in a good health.

אחרת, תחזיר False.

Ocypode:

מחלקה אשר מייצגת סרטן מטיפוס Ocypode המיוצג על ידי השדות הבאים:

- **name**- משתנה מטיפוס מחרוזת לא ריקה המייצגת את שם החיה.
- **age**- משתנה מטיפוס int, מספר שלם הגדול מ-0 וקטן מ-120 המייצג את גיל החיה.
- **food**- משתנה מטיפוס int, המייצג את כמות האוכל הזמינה לחיה. כאשר נוצר מופע חדש של Ocypode, ערכו של המשתנה food מאותחל ב-10.
- **width**- משתנה מטיפוס int המייצג את רוחב החיה. רוחב ה-Ocypode מאותחל ב-7.
- **height**- משתנה מטיפוס int המייצג את גובה החיה. גובה ה-Ocypode מאותחל ב-4.

- **directionH**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האופקי של ה-Ocypode באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- שמאלה, 1- ימינה.
- **x**- משתנה מטיפוס int המייצג את מיקום ה-Ocypode בציר ה-x באקווריום, מספר שלם הגדול מ-0.
- **y**- משתנה מטיפוס int המייצג את מיקום ה-Ocypode בציר ה-y באקווריום, מספר שלם הגדול מ-0.

ממשו את מחלקת Ocypode:

def __init__(self, name, age, x, y, directionH):

בנאי המאתחל את שדות המחלקה.

במידה ואחד השדות קיבל ערך לא תקין יש לזרוק חריגה מתאימה.

def __str__(self):

על השיטה להחזיר מחרוזת המייצגת את סרטן ה-Ocypode בפורמט:

The ocypode name is age years old and has food food.

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

לדוגמא:

The ocypode myocypode is 14 years old and has 10 food.

def get_animal(self):

השיטה מחזירה רשימה מקוננת המכילה את צורת סרטן ה-Ocypode בכיוון ההתקדמות שלו.

לדוגמא עבור סרטן Ocypode המתקדם בכיוון ימין תוחזר הרשימה הבאה (שימו לב שעבור Ocypode אין השפעה של כיוון ההתקדמות על צורתו וצורתו זהה באשר הוא נע ימינה או שמאלה).

```
[[" ", "*", " ", " ", " ", "*", " "],
 [" ", " ", "*", "*", "*", " ", " "],
 ["*", "*", "*", "*", "*", "*", "*"],
 ["*", " ", " ", " ", " ", " ", "*"]]
```

def __repr__ (self):

השיטה מחזירה מחרוזת על פי הרשימה המקוננת המתקבלת מהשיטה get_animal. חיבור מחרוזת הפלט יתבצע ע"י חיבור איברי הרשימה הפנימית מופרדים אחד מהשני בתו ריק. בנוסף, בסוף כל רשימה פנימית יוסף למחרוזת '\n' כלומר, שורה חדשה.

לדוגמא עבור סרטן ה-Ocypode תוחזר המחרוזת הבאה:

def get_position (self):

השיטה מחזירה טאפל (tuple) המייצג את מיקום סרטן ה-Ocypode באופן הבא: (x,y).

def get_directionH (self):

השיטה מחזירה את כיוון ההתקדמות בציר האופקי של סרטן ה-Ocypode.

def get_size (self):

השיטה מחזירה טאפל (tuple) המייצג את גודל סרטן ה-Ocypode באופן הבא: (width,height).

def dec_food (self):

השיטה מעדכנת את כמות האוכל הזמינה לסרטן ה-Ocypode ומחסירה ממנו מנה אחת.

def add_food(self, amount):

השיטה מעדכנת את כמות האוכל הזמינה לסרטן ה-Ocypode ומעלה אותו ב-amount כך ש-amount הינו משתנה מטיפוס int הגדול מ-0. (ניתן להניח כי הקלט תקין).

def inc_age (self):

השיטה מעדכנת את הגיל של ה-Ocypode ומעלה אותו ב-1.

def move (self):

השיטה מזיזה את סרטן ה-Ocypode צעד אחד לפי כיוון ההתקדמות שלו. בכל צעד סרטן ה-Ocypode מתקדם יחידה אחת בציר האופקי. כיוון ההתקדמות נקבע על פי הערך בשדה directionH והצעד מתבצע על ידי שינוי הערך בשדה x כנדרש.

התדקמות שמאלה = הקטנת הערך בציר ה-x ביחידה.

התקדמות ימינה = הגדלת הערך בציר ה-x ביחידה.

* שימו לב שבשלב זה אין צורך לדאוג לכך שהערכים ב-x ו-y עלולים להיות שליליים.

def set_directionH(self, directionH):

השיטה מעדכנת את כיוון ההתקדמות של סרטן ה-Ocypode בהתאם למשתנה הקלט directionH (ניתן להניח ש-directionH הינו מטיפוס int השווה ל-0 או 1).

def starvation (self):

השיטה בודקת האם סרטן ה-Ocypode גווע מרעב על ידי בדיקת כמות האוכל הזמינה שנותרה לו. במידה וכמות האוכל של סרטן ה-Ocypode הינה 0 השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died at the age of age years because it ran out of food.

אחרת, תחזיר False.

def die (self):

השיטה בודקת האם סרטן ה-Ocypode מת בשיבה טובה על ידי בדיקת גילו. במידה וגילו הינו 120 ומעלה, השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died in a good health.

אחרת, תחזיר False.

Shrimp:

מחלקה אשר מייצגת סרטן מטיפוס Shrimp המיוצג על ידי השדות הבאים:

- **name**- משתנה מטיפוס מחרוזת לא ריקה המייצגת את שם החיה.
- **age**- משתנה מטיפוס int, מספר שלם הגדול מ-0 וקטן מ-120 המייצג את גיל החיה.
- **food**- משתנה מטיפוס int, המייצג את כמות האוכל הזמינה לחיה. כאשר נוצר מופע חדש של Shrimp, ערכו של המשתנה food מאותחל ב-10.
- **width**- משתנה מטיפוס int המייצג את רוחב החיה. רוחב ה-Shrimp מאותחל ב-7.
- **height**- משתנה מטיפוס int המייצג את גובה החיה. גובה ה-Ocypode מאותחל ב-3.

- **directionH**- משתנה מטיפוס int המייצג את כיוון ההתקדמות בציר האופקי של ה-Shrimp באקווריום. ערך זה יכול להיות שווה ל-0 או 1 בלבד. 0- שמאלה, 1- ימינה.
- **x**- משתנה מטיפוס int המייצג את מיקום ה-Shrimp בציר ה-x באקווריום, מספר שלם הגדול מ-0.
- **y**- משתנה מטיפוס int המייצג את מיקום ה-Shrimp בציר ה-y באקווריום, מספר שלם הגדול מ-0.

ממשו את מחלקת shrimp:

def __init__(self, name, age, x, y, directionH):

בנאי המאתחל את שדות המחלקה.

במידה ואחד השדות קיבל ערך לא תקין יש לזרוק חריגה מתאימה.

def __str__(self):

על השיטה להחזיר מחרוזת המייצגת את סרטן ה-Shrimp בפורמט:

The shrimp name is age years old and has food food.

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

לדוגמא:

The shrimp myshrimp is 15 years old and has 10 food.

def get_animal(self):

השיטה מחזירה רשימה מקוננת המכילה את צורת סרטן ה-Shrimp בכיוון ההתקדמות שלו. שימו לב כי צורת סרטן ה-Shrimp נקבעת לפי כיוון ההתקדמות בציר האופקי (ימינה/שמאלה).

לדוגמא עבור סרטן Shrimp המתקדם בכיוון ימין תוחזר הרשימה הבאה:

```
[[" ", " ", " ", " ", "*", " ", "*"],
 ["*", "*", "*", "*", "*", "*", " "],
 [" ", " ", "*", " ", "*", " ", " "]]
```

def __repr__ (self):

השיטה מחזירה מחרוזת על פי הרשימה המקוננת המתקבלת מהשיטה get_animal. חיבור מחרוזת הפלט יתבצע ע"י חיבור איברי הרשימה הפנימית מופרדים אחד מהשני בתו ריק. בנוסף, בסוף כל רשימה פנימית יוסף למחרוזת '\n' כלומר, שורה חדשה.

לדוגמא עבור סרטן ה-Shrimp המתקדם בכיוון ימין תוחזר המחרוזת הבאה:

def get_position (self):

השיטה מחזירה טאפל (tuple) המייצג את מיקום סרטן ה-Shrimp באופן הבא: (x,y).

def get_directionH (self):

השיטה מחזירה את כיוון ההתקדמות בציר האופקי של סרטן ה-Shrimp.

def get_size (self):

השיטה מחזירה טאפל (tuple) המייצג את גודל סרטן ה-Shrimp באופן הבא: (width,height).

def dec_food (self):

השיטה מעדכנת את כמות האוכל הזמינה לסרטן ה-Shrimp ומחסירה ממנו מנה אחת.

def add_food(self, amount):

השיטה מעדכנת את כמות האוכל הזמינה לסרטן ה-Shrimp ומעלה אותו ב-amount כך ש-amount הינו משתנה מטיפוס int הגדול מ-0. (ניתן להניח כי הקלט תקין).

def inc_age (self):

השיטה מעדכנת את הגיל של ה-Shrimp ומעלה אותו ב-1.

def move (self):

השיטה מזיזה את סרטן ה-Shrimp צעד אחד לפי כיוון ההתקדמות שלו. בכל צעד, סרטן ה-Shrimp מתקדם יחידה אחת בציר האופקי. כיוון ההתקדמות נקבע על פי הערך בשדה directionH והצעד מתבצע על ידי שינוי הערך בשדה x בנדרש.

התדקמות שמאלה = הקטנת הערך בציר ה-x ביחידה.

התקדמות ימינה = הגדלת הערך בציר ה-x ביחידה.

* שימו לב שבשלב זה אין צורך לדאוג לכך שהערכים ב-x ו-y עלולים להיות שליליים.

def set_directionH(self, directionH):

השיטה מעדכנת את כיוון ההתקדמות של סרטן ה-Shrimp בהתאם למשתנה הקלט directionH (ניתן להניח ש-directionH הינו מטיפוס int השווה ל-0 או 1).

def starvation (self):

השיטה בודקת האם סרטן ה-Shrimp גווע מרעב על ידי בדיקת כמות האוכל הזמינה שנותרה לו. במידה וכמות האוכל של סרטן ה-Shrimp הינה 0 השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died at the age of age years because it ran out of food.

אחרת, תחזיר False.

def die (self):

השיטה בודקת האם סרטן ה-Shrimp מת בשיבה טובה על ידי בדיקת גילו. במידה וגילו הינו 120 ומעלה, השיטה תחזיר True ותדפיס את ההודעה הבאה:

name died in a good health.

אחרת, תחזיר False.

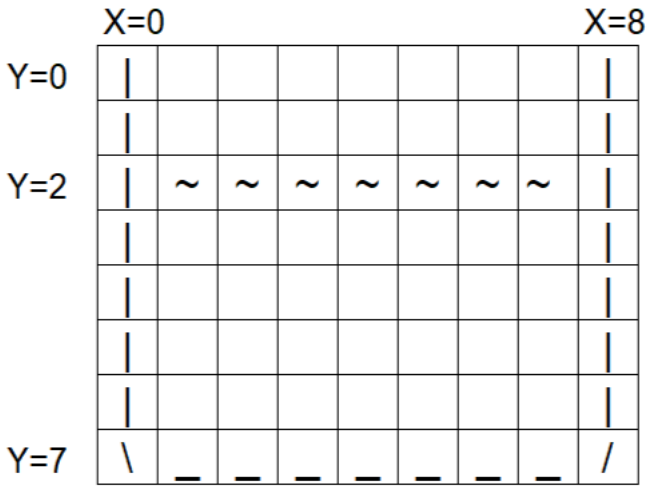
Aquarium:

מחלקה אשר מייצגת אקווריום. במחלקת האקווריום ישמרו החיות הנמצאות בו ויתאפשר ממשק לניהול חיות האקווריום. האקווריום משתנה בכל סבב (המתאר יחידת זמן): החיות נעות, מזדקנות ואוכלות. כל סבב נקרא צעד (step). מחלקת האקווריום מאפשרת ממשק משתמש גרפי, המאפשר למשתמש לצפות באקווריום ובחיות הנמצאות בו, כפי שראיתם בסרטון בתחילת התרגיל וכן בדוגמאות בהמשך המחלקה.

אקווריום מתואר על ידי השדות הבאים:

- **aqua_height**- משתנה מטיפוס int המייצג את ממד הגובה של האקווריום וערכו גדול שווה ל-25. במידה והוכנס מספר הקטן מ-25, תזרק חריגה מסוג TooSmallAquariumSize.
- **aqua_width** - משתנה מטיפוס int המייצג את ממד הרוחב של האקווריום וערכו גדול שווה ל-40. במידה והוכנס מספר הקטן מ-40, תזרק חריגה מסוג TooSmallAquariumSize.
- **step**- משתנה מטיפוס int המייצג את מספר הסבב (זמן). ביצירת האקווריום הזמן הינו 0.

- **animals**- משתנה מטיפוס רשימה המייצגת את החיות באקווריום. כאשר האקווריום נוצר הרשימה ריקה.
- **board**- משתנה מטיפוס רשימה מקוננת המייצג את האקווריום באופן הבא (דוגמא עבור aqua_height=8, aqua_width=9):



  - ציר ה-X הינו משמאל לימין. ציר ה-Y הינו מלמעלה למטה.
  - קו המים תמיד יהיה בעמודה השלישית מלמעלה (Y=2) ללא קשר לגודל האקווריום.
  - שימו לב שפינות האקווריום אינן קו ישר.

ייצוג האקווריום לדוגמא באשר aqua_height=7, aqua_width=8 על ידי רשימה מקוננת:

```
[['|',' ',' ',' ',' ',' ',' ',' ','|']
['|',' ',' ',' ',' ',' ',' ',' ','|']
['|','~','~','~','~','~','~','~','|']
['|',' ',' ',' ',' ',' ',' ',' ','|']
['|',' ',' ',' ',' ',' ',' ',' ','|']
['|',' ',' ',' ',' ',' ',' ',' ','|']
['\\','_','_','_','_','_','_','_','/']]
```

שימו לב כי בכל תא ריק קיים רווח.

ממשו את מחלקת aquarium:

def __init__(self, aqua_width, aqua_height):

בנאי המאתחל את שדות המחלקה.

במידה ואחד השדות קיבל ערך לא תקין יש לזרוק חריגה מתאימה.

def __str__(self):

השיטה תחזיר מחרוזת המייצגת aquarium בפורמט:

The aquarium, sized height/width and currently in step step, contains the following animals:

כאשר אדום מסמן את התווים הקבועים ושחור את ערכי השדות המתאימים.

לאחר מכן בשורה חדשה יופיעו כל אחת מהחיות באקווריום לפי סדר ההכנסה (כאשר הן הוכנסו לשדה animals).

לדוגמא:

The aquarium, sized 25/40 and currently in step 0, contains the following animals:

The ocypode myocypode is 14 years old and has 10 food.

The shrimp myshrimp is 15 years old and has 10 food.

The scalar myscalar is 12 years old and has 10 food.

The molly mymolly is 13 years old and has 10 food.

def __repr__(self):

השיטה מחזירה מחרוזת על פי הרשימה המקוננת בשדה board באופן הבא:

כל איבר ברשימה יתווסף למחרוזת כך שאחריו תו ריק (גם אחרי איבר שהוא רווח יתווסף רווח).

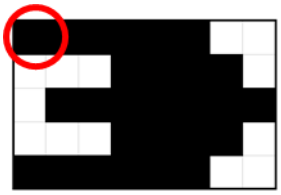
בנוסף, בסוף כל רשימה פנימית יוסף למחרוזת '\n' כלומר, שורה חדשה (ולא יופיע רווח לפני השורה החדשה).

לדוגמא עבור אקווריום ריק:

def feed_all(self):

השיטה תוסיף לכל החיות באקווריום 10 יחידות לכמות האוכל הזמינה לכל אחד.

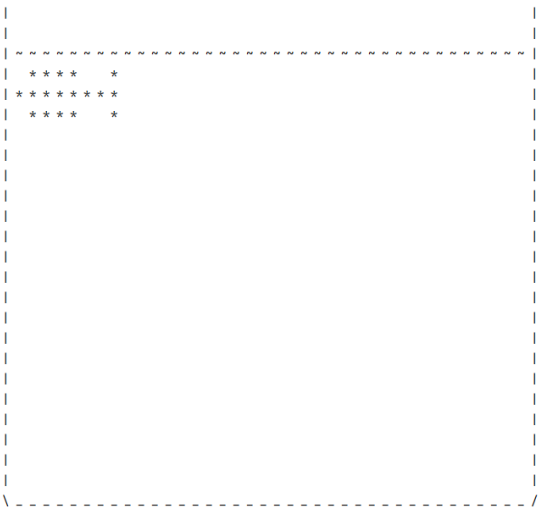
def __insert_animal_to_board (self, animal):



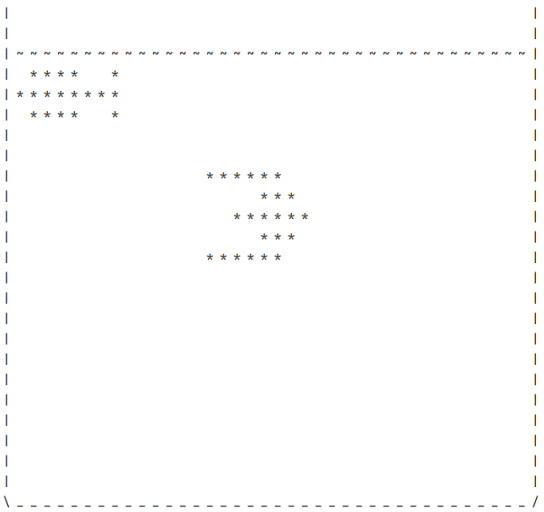
שיטה פרטית של המחלקה. השיטה מוסיפה ללוח (לשדה board) את החיה animal לפי הרשימה המקוננת המתקבלת מהשיטה get_animal של החיה. הוספת החיה לשדה board מתבצעת ע"י מיקום החיה לפי השדות x ו-y של החיה שמייצגים את התא הימני העליון ([0][0]) ברשימה המקוננת המייצגת את החיה. ניתן להניח כי המיקום תקין (נמצא במקום תקין בלוח כדי להכיל את החיה בשלמותה- הסבר על כך יורחב בהמשך).

שימו לב כי הוספת החיה על ידי שיטה זו הינה לשדה board בלבד ולא משנה את הרשימה שבשדה animals.

לדוגמא, לאחר הוספת חיה מסוג Molly עם כיוון שחיה שמאלה במיקום (1,3) לוח האקווריום יתעדכן באופן הבא (ייצוג ב-repr של האקווריום):



לאחר הוספת חיה נוספת מסוג Scalar במיקום (15,8):

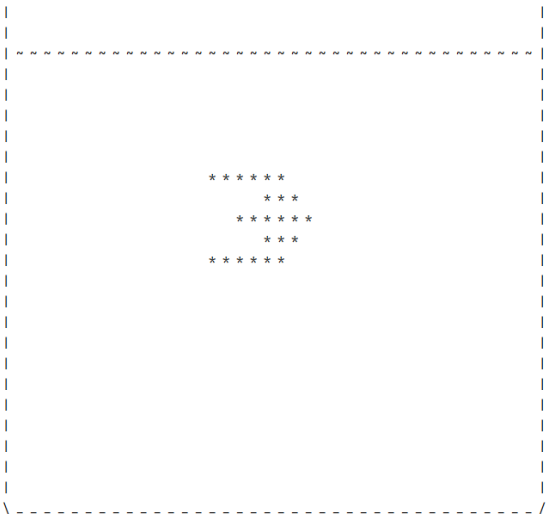


def __delete_animal_from_board (self, animal):

שיטה פרטית של המחלקה. השיטה מוחקת מהלוח (מהשדה board) את החיה animal לפי מיקום החיה וגודלה.

שימו לב כי מחיקת החיה על ידי שיטה זו הינה מהשדה board בלבד ולא משנה את הרשימה שבשדה Animals.

לדוגמא, לאחר מחיקת חיה מסוג Molly במיקום (1,3) לוח האקווריום מהדוגמא האחרונה יתעדכן באופן הבא (ייצוג ב-repr של האקווריום):



- שימו לב- במידה ויש שתי חיות חופפות במרחב האקווריום, מחיקת חיה אחת תגרום למחיקת חלק מהחיה האחרת בתצוגה. אתם לא נדרשים לפתור את הבעיה הזו בשיטה זאת.

def add_animal(self, name, age, x, y, directionH, directionV, animaltype):

השיטה מוסיפה חיה לאקווריום, כלומר מוסיפה אותה לשדה board ולרשימת החיות בשדה animals.

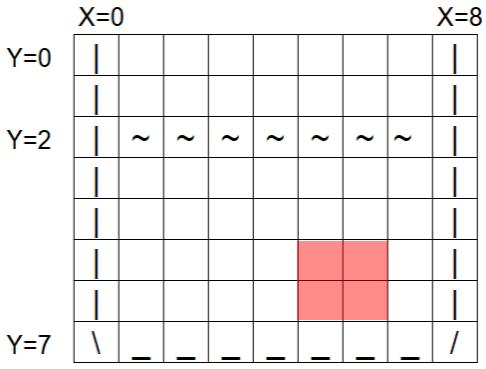
טיפ: מומלץ לחלק את השיטה למספר שיטות כאשר לכל אחת יש תפקיד אחר.

השיטה מקבלת כקלט:

- animalytpe משתנה מטיפוס מחרוזת המייצג את סוג החיה. ערכו של המשתנה animalytpe יכול להיות שווה לאחד מארבעת האפשרויות: scalar, molly, ocypode, shrimp. במידה והוכנסה מחרוזת אחרת יש לזרוק חריגה מסוג InvalidAnimalType עם המחרוזת אותה ניסו להכניס.
- x ו-y מייצגים את מיקום החיה המבוקש. ניתן להניח כי הוכנסו מספרים שלמים. במידה והמיקום חורג מגבולות האקווריום (שימו לב שלא ניתן למקם חיה על דפנות האקווריום או

על ומעל לקו המים), יש למקם את החיה בנקודה הכי קרובה למיקום שנמצאת צמוד לדופן האקווריום/ הקרקעית/ קו המים. יש דוגמאות בהמשך.

- סרטנים לא יכולים לשחות ולכן תמיד ימוקמו על הקרקעית (ללא תלות בערך ה-y). לדוגמא סרטן בגודל 2X2 אשר ערך ה-x שנתנו עבורו הינו 5 יתמקם בלוח הבא במיקום הבא:

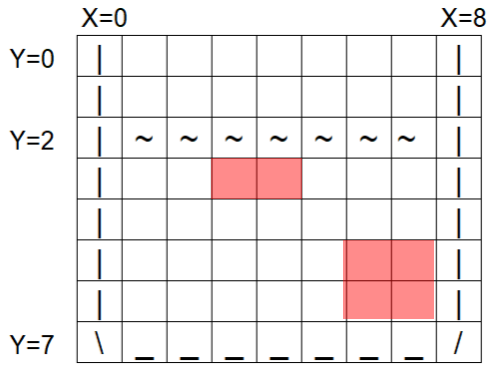


- דגים שוחים מעל הגובה המקסימלי של הסרטנים שמוגדר להיות 4 יחידות מעל קרקעית האקווריום. כלומר, אם גובה האקווריום מוגדר ל-25, דגים יכול להיות החל מ-y=20 (שימו לב שמיקום הדג נקבע על פי הפינה השמאלת העליונה לכן צריך לחשב את ערך ה-y של הדג כאשר בולו (לפי השיטה repr שלו) לא יעבור את y=20. עבוד דג בגובה 5, המיקום המקסימלי שיכול להיות עבורו בציר ה-y הינו 15.

דוגמא עבור דג Scalar:

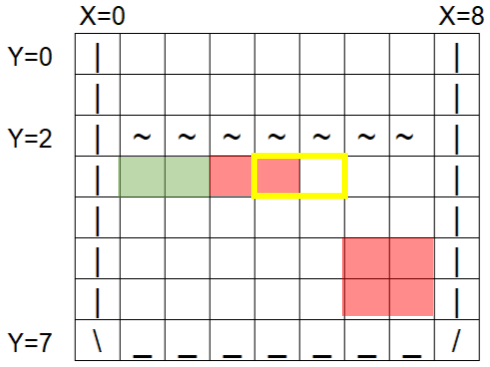
- חיות לא יכולות להיות ממוקמות במיקום שיש בו חיה אחרת. לכן, יש לבדוק האם המקום פנוי לחלוטין ובמידה ולא לזרוק חריגה מסוג NotAvailablePlace. מקום פנוי לחלוטין הינו מקום בו החיה יכולה להתמקם במלואה לפי השיטה repr מבלי לחפוף לחיה אחרת.

לדוגמא:



עבור הלוח הנ"ל, הוספת חיה חדשה בגודל 1X2 יכולה להתבצע במיקום (1,3) אך לא יכולה להתבצע במיקום (4,3).

בירוק הוספה למיקום הפנוי, במסגרת צהובה למקום הלא פנוי:

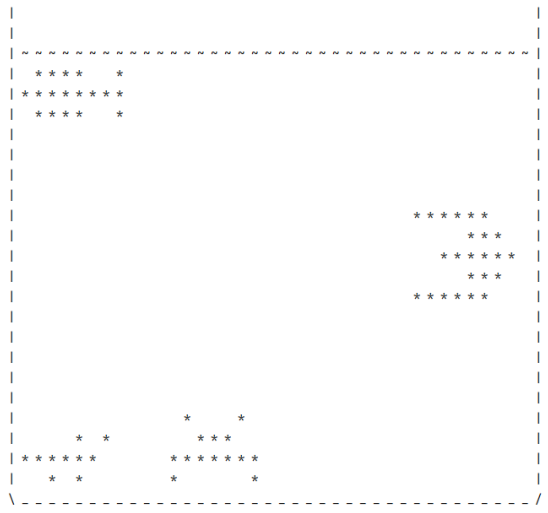


- directionH ו-directionV הינם כיוון החיה בציר האופקי והאנכי בהתאם. שימו לב שאין משמעות לציר האנכי עבור סרטנים.

<u>דוגמא 1:</u> עבור קטע הקוד הבא:

```
aqua = Aquarium(40, 25)
aqua.add_animal('myocypode', 14, 12, 12, 0, 0, 'ocypode')
aqua.add_animal('myshrimp', 15, 1,3,1,0,'shrimp')
aqua.add_animal('myscalar', 17, 30,10,1,0,'scalar')
aqua.add_animal('mymolly', 13, 1,3,0,0,'molly')
print(repr(aqua))
```

יתקבל הלוח הבא:

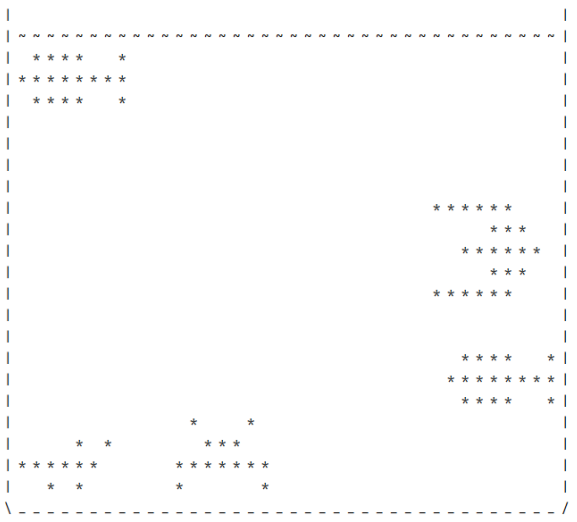


שימו לב כי הסרטנים צמודים לקרקעית למרות שציר ה-y שהוזן להם שונה.

<u>דוגמא 2:</u> כעת נוסיף חיה נוספת כאשר ציר ה-x המוזן הינו 100 וציר ה-y המוזן הינו 100:

```
aqua.add_animal('mymolly2', 13, 100,100,0,0,'molly')
```

הלוח החדש שיתקבל הינו:

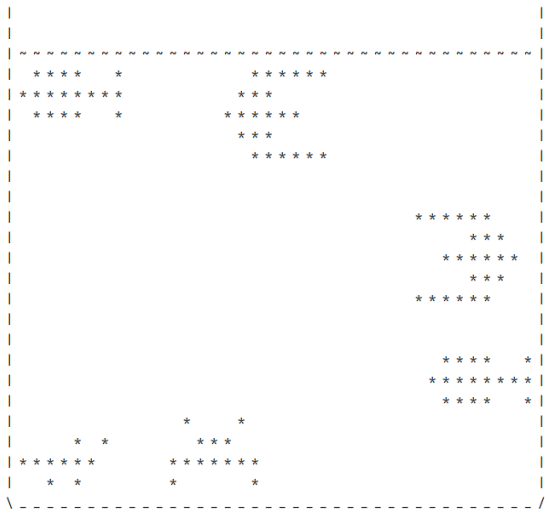


שימו לב כי דג ה-Molly החדש התווסף למיקום הכי נמוך וצמוד לדופן הימנית מכיוון ש-100 מחוץ לתחום בשני הצירים.

<u>דוגמא 3:</u> כעת נוסיף חיה נוספת כאשר ציר ה-x המוזן הינו 16 וציר ה-y המוזן הינו 1:

```
aqua.add_animal('myscalar2', 17, 16,1,0,1,'scalar')
```

הלוח החדש שיתקבל הינו:



שימו לב כי למרות שהוזן בציר ה-y 1, החיה החדשה מוקמה במיקום הכי גבוה מתחת לפני המים שהינם בגובה 2 (בלומר ב-y=3).

<u>דוגמא 4:</u> בעת נוסיף חיה נוספת באשר ציר ה-x המוזן הינו 5 וציר ה-y המוזן הינו 5:

```
aqua.add_animal('myscalar3', 17, 5,5,0,1,'scalar')
```

תתקבל הודעת שגיאה:

```
Exceptions.NotAvailablePlace
```

מכיוון שכבר יש חיה במיקום זה.

<u>דוגמא 5:</u> בעת נוסיף חיה נוספת שסוג החיה אינו מהרשימה המותרת:

```
aqua.add_animal('mywhale', 17, 5,5,0,1,'whale')
```

תתקבל הודעת השגיאה הבאה:

```
Exceptions.InvalidAnimalType: Error: "whale" is an invalid animal type. The valid animal types are: molly, scalar, ocypode, shrimp.
```

def __kill_animal(self, animal):

שיטה פרטית של המחלקה. השיטה מקבלת חיה ובודקת האם היא גוועה מרעב/ או שהיא מתה בשיבה טובה. במידה וכן, השיטה מדפיסה את ההודעה המתאימה כפי שפורטו בשיטות המתאימות בחיות, מוחקת את החיה מהלוח (מהשדה board) ומסירה אותה מרשימת החיות של האקווריום (מהשדה animals).

מכיוון ששיטה זו הינה פרטית, ניתן להניח כי הוכנסה חיה חוקית אשר קיימת באקווריום.

def next_step(self):

שיטה המבצעת סבב (צעד) באקווריום כהלכה. כאשר מתבצע סבב באקווריום כל החיות זזות בכיוון ההתקדמות שלהן צעד 1.

טיפ: מומלץ לחלק את המימוש למספר שיטות כאשר לכל אחת יש תפקיד אחר (מודולריות).

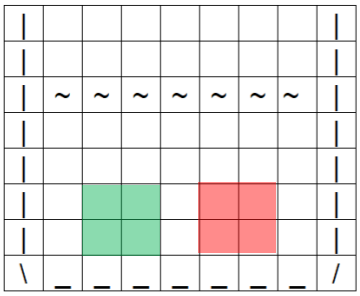
עליכם יהיה לממש את הפעולות הבאות:

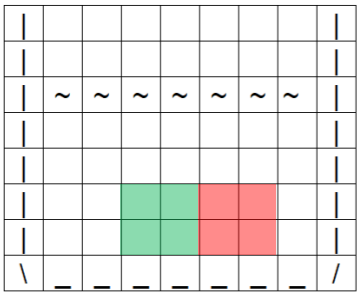
תחילה נגדיל את ערכו של השדה step ביחידה אחת.

לאחר מכן, נבדוק איזה מהחיות גוועו ברעב/ או מתו בשיבה טובה. ונוציא אותן מהאקווריום בהתאם.

בשלב הבא נבצע את הצעד של החיות באקווריום. כל חיה (על פי סדר ההוספה לאקווריום) תבצע תזוזה אחת (move) על פי החוקים הבאים:

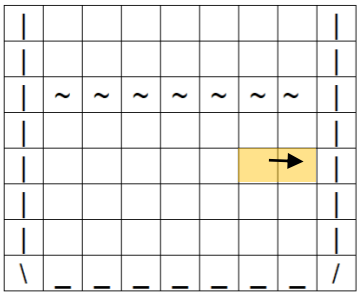
- בעת תזוזתם של הדגים, הם יכולים לחפוף אחד את השני (דמיינו שהאקווריום עמוק והם שוחים בעומקים שונים). שימו לב שלמרות זאת עדיין יש להניח כי לא ניתן להוסיף דג במקום התפוס על ידי דג אחר.
- כאשר סרטנים עלולים להתנגש האחד בשני, שניהם ישנו את כיוון ההתקדמות שלהם ויבצעו תזוזה אחת בכיוון החדש.
  - סרטנים מוגדרים **כעלולים להתנגש** כאשר המיקום שלהם לפני התזוזה החדשה הינו בתאים צמודים או שיש ביניהם רווח אחד, כלומר במידה ויתקדמו כל אחד צעד לקראת השני הם יעלו אחת על השני. לדוגמא: (כאשר הריבועים הצבעוניים מייצגים סרטנים)



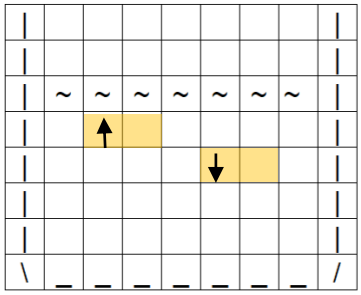


  - ניתן להניח כי רוחב סרטן מקסימלי הינו 7 וניתן להעריך את מיקום הסרטן לפיו ללא קשר לרוחב האמיתי שלו.
  - הסרטנים יוגדרו **כמתנגשים** אם ורק אם הם עלולים להתנגש (על פי ההגדרה בסעיף הקודם) וכיוון ההתקדמות שלהם הוא האחד כלפי השני.
  - במידה ונמצא זוג סרטנים מתנגשים יש לשנות את כיוון ההתקדמות של שניהם ולבצע תזוזה אחת עבור כל אחד בכיוון החדש.

- כאשר חיה מתנגשת בקיר האקווריום היא משנה את כיוון ההתקדמות שלה בציר האופקי, ומבצעת תזוזה אחת בכיוון החדש. חיה מוגדרת כמתנגשת בקיר במידה ובצעד הבא היא תחפוף את קיר האקווריום. לדוגמא:



- כאשר דג מתנגש בגובה המים או מתנגש בגובה המינימלי המותר (כפי שהוסבר בשיטה add_animal) הוא ישנה את כיוון ההתקדמות שלו בציר האנכי ויבצע תזוזה אחת בכיוון החדש. דג מוגדר כמתנגש בגובה המים אם בצעד הבא היא יחפוף את גובה המים. דג מוגדר כמתנגש בגובה המינימלי המותר אם בצעד הבא הוא יעבור אותו. לדוגמא: (בדוגמא זו הגובה המינימלי המותר הוגדר ל-2 כלומר כאשר y=7 הוא יכול להיות ממוקם ב-y=4 במקסימום, כיוון ההתקדמות בציר האנכי מסומן על ידי חץ)



- במידה והחיה לא מתנגשת בשום דבר והיא יכול להתקדם בחופשיות תבצע תזוזה אחת.

לאחר הצעד נבדוק האם יש להוריד את כמות האוכל או להגדיל את גיל החיות על פי החוקיות הבאה:

- בכל 10 צעדים כמות האוכל יורדת לכל החיות ביחידה אחת.
- בכל 10 צעדים גיל החיות גדל ב-1.

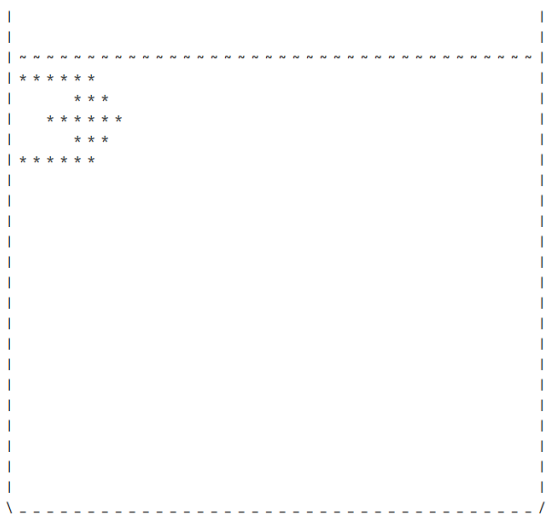
שימו לב, האוכל והגיל מתעדכנים לכל החיות באותו הצעד ולא לכל חיה לפי הזמן בה היא נכנסה. לדוגמה – כמות האוכל של כל החיות יורדת בתור מספר 10 , 20 וכו' (הצעד הראשון הוא מספר 1). הכוונה – כאשר מתבצע צעד כדאי למחוק את כל החיות מלוח האקווריום (השדה Board) ולהוסיפם מחדש ללוח לאחר עדכון המיקומים החדשים של כל החיות.

דוגמא 1: שחייה של דג

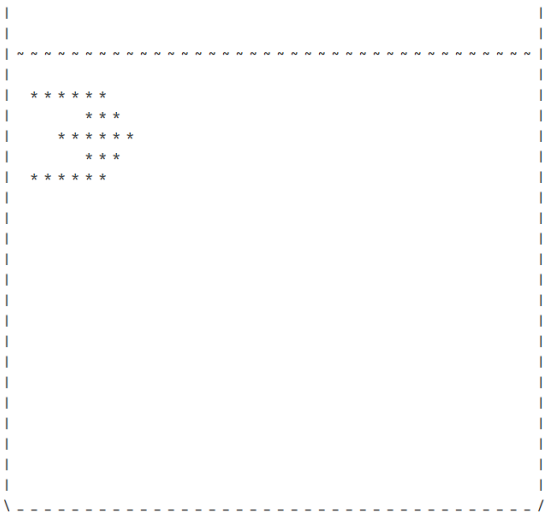
```
aqua = Aquarium(40, 25)
aqua.add_animal('myscalar', 17, 1,3,1,0,'scalar')
```

שימו לב שכיוון השחיה של הדג הינו ימינה (1) ולמטה (0)

מצב הלוח ההתחלתי:

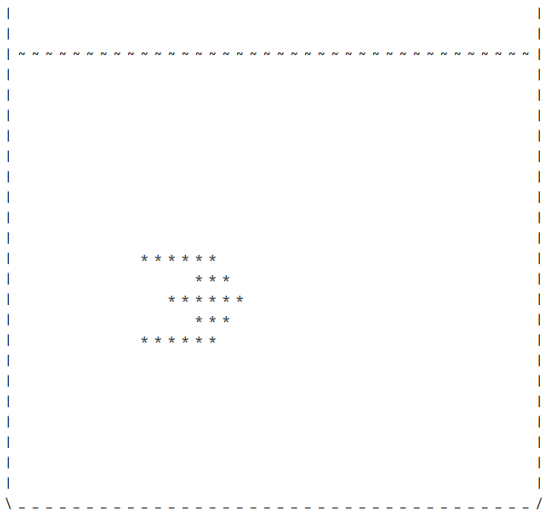


לאחר צעד 1:



ניתן לראות שהדג התקדם בכיוון השחיה צעד אחד, כלומר צעד אחד ימינה ואחד למטה.

לאחר 8 צעדים נוספים:

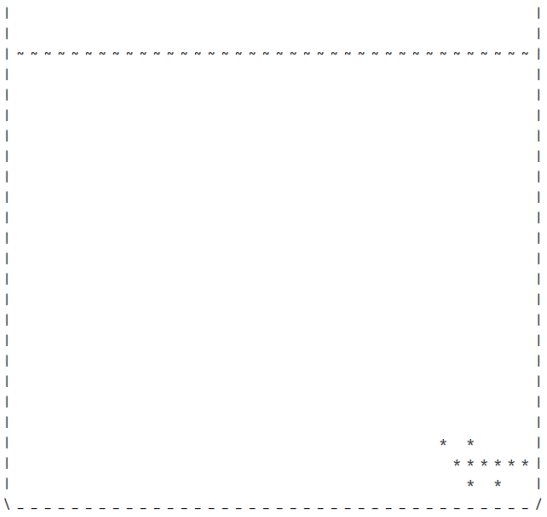


דוגמא 2: הליכה של סרטן

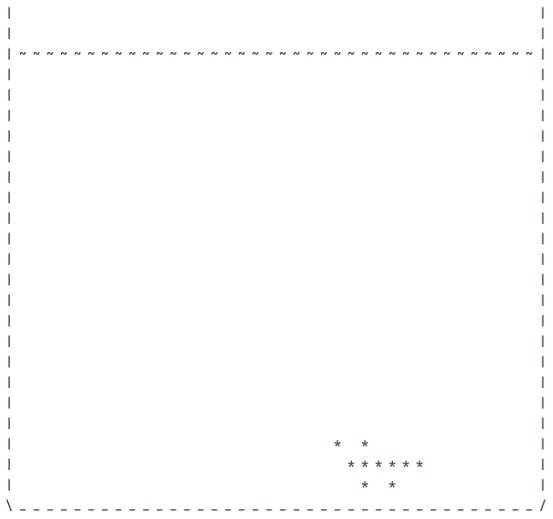
```
aqua = Aquarium(40, 25)
aqua.add_animal('myshrimp', 15, 32,1,0,0,'shrimp')
```

כיוון ההליכה של הסרטן הינה שמאלה.

מצב הלוח ההתחלתי:



מצב הלוח לאחר 8 צעדים:

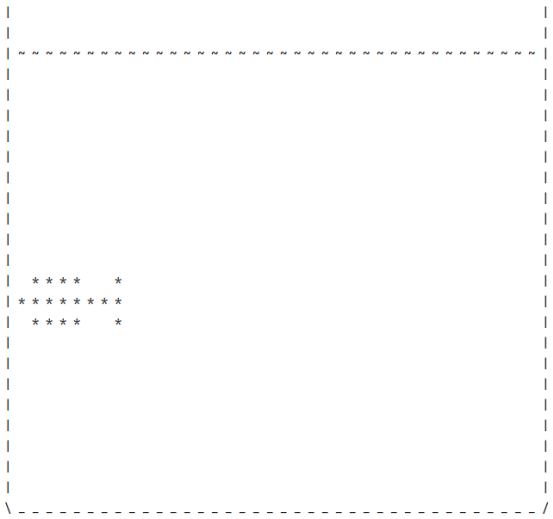


דוגמא 3: התנגשות חיה בקיר האקווריום:

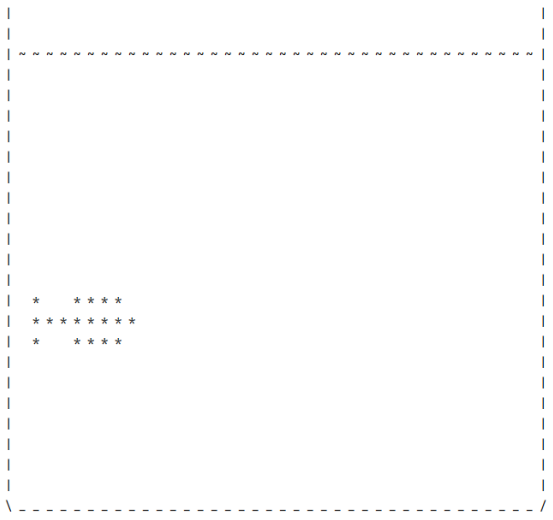
```
aqua = Aquarium(40, 25)
aqua.add_animal('mymolly', 13, 1,13,0,0,'molly')
```

כיוון שחיית הדג הינו שמאלה ולמטה.

הלוח ההתחלתי:



לאחר צעד 1:



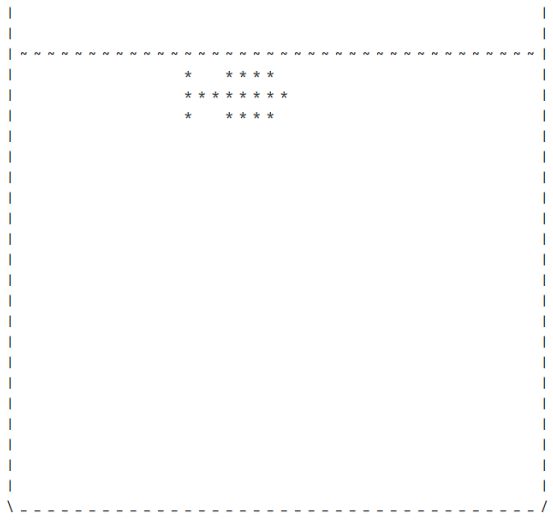
הדג שינה את כיוון השחייה האופקי שלו (משמאלה לימינה) וביצע צעד אחד בכיוון החדש (ימינה ולמטה).

<u>דוגמא 4: התנגשות חיה בגובה המים המותר:</u>

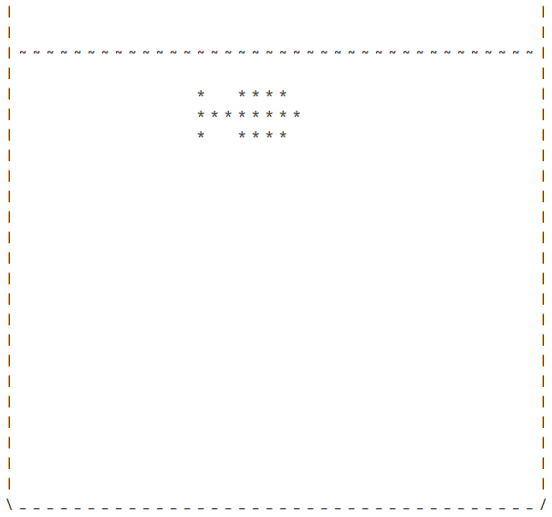
```
aqua = Aquarium(40, 25)
aqua.add_animal('mymolly', 13, 13,3,1,1,'molly')
```

כיוון השחייה של הדג הינו למעלה וימינה.

מצב הלוח ההתחלתי:



מצב הלוח לאחר צעד 1:

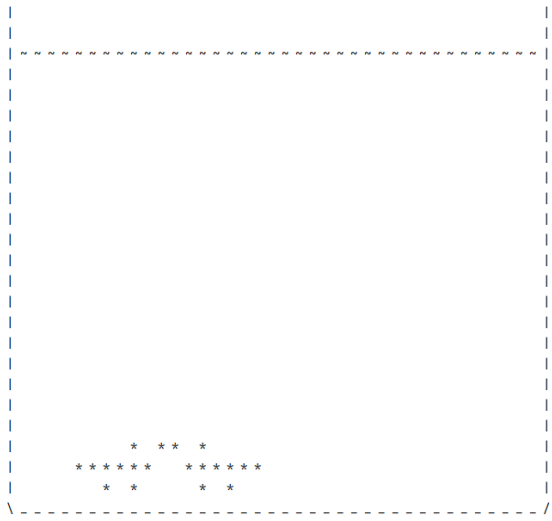


הדג שינה את כיוון השחיה האנכי שלו מלמעלה למטה וביצע צעד אחד בכיוון השחייה החדש (למטה וימינה).

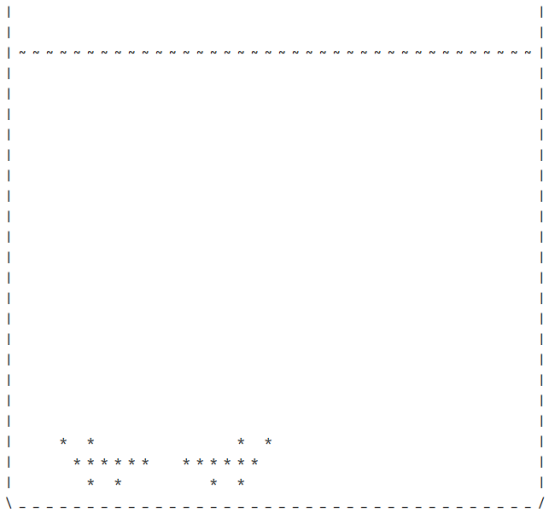
<u>דוגמא 5: התנגשות סרטנים אחד בשני:</u>

```
aqua = Aquarium(40, 25)
aqua.add_animal('myshrimp1', 14, 12, 12, 0, 0, 'shrimp')
aqua.add_animal('myshrimp2', 15, 5,3,1,0,'shrimp')
```

מצב הלוח התחלתי:



מצב הלוח לאחר צעד 1:



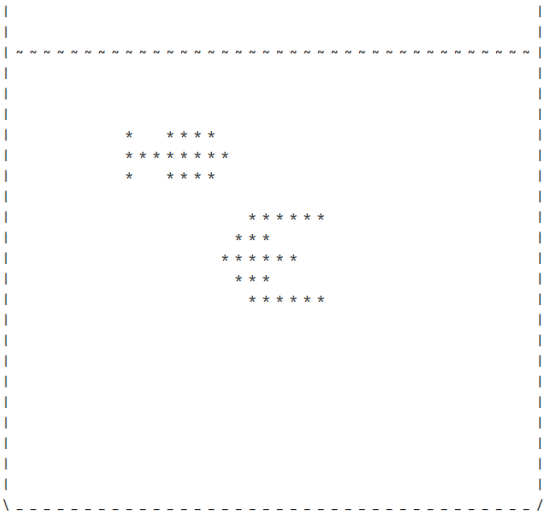
שני הסרטנים שינו את כיוונים האופקי וביצעו צעד אחד בכיוון החדש.

<u>דוגמא 6: דגים השוחים אחד על השני:</u>

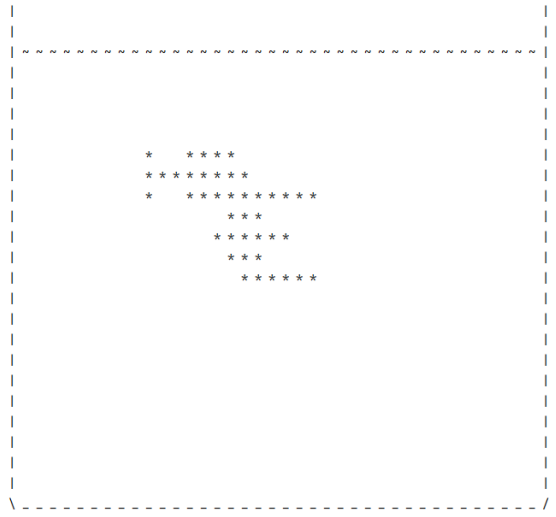
```
aqua = Aquarium(40, 25)
aqua.add_animal('myscalar', 17, 16,10,0,1,'scalar')
aqua.add_animal('mymolly', 13, 9,6,1,0,'molly')
```

כפי שניתן לראות, כיווני הדגים מנוגדים.

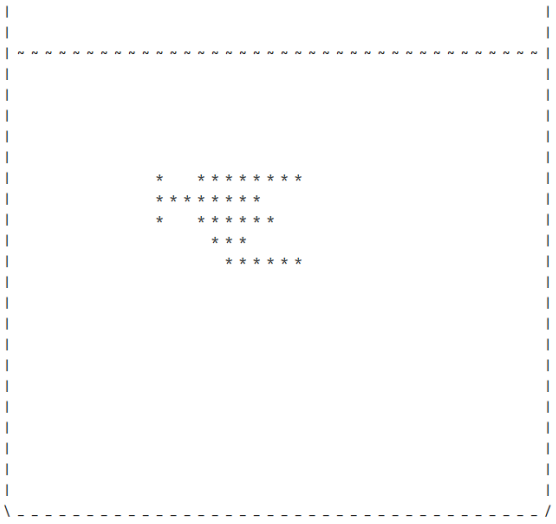
מצב הלוח ההתחלתי:



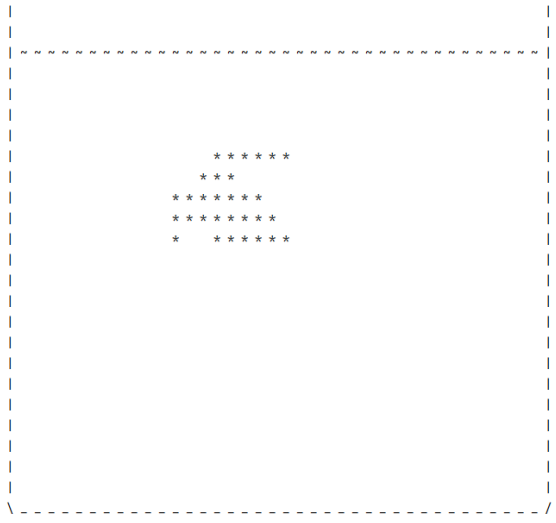
לאחר צעד 1:



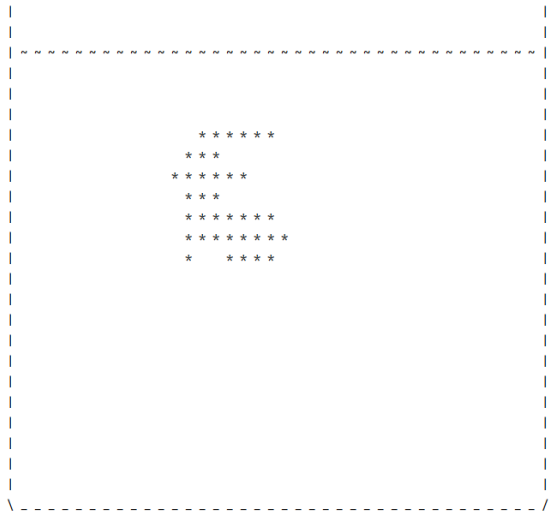
לאחר צעד 2:



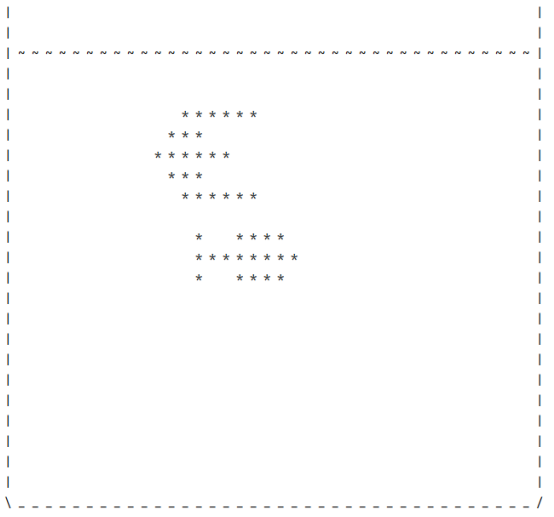
לאחר צעד 3:



לאחר צעד 4:



לאחר צעד 5:



def several_steps(self, steps):

השיטה מבצעת מספר צעדים באקווריום.

מספר הצעדים יתקבל כארגומנט וניתן להניח כי הוא תקין (מספר שלם הגדול מ-0)

דוגמא:

```
aqua = Aquarium(40, 25)
aqua.add_animal('myocypode', 118, 19, 12, 0, 0, 'ocypode')
aqua.add_animal('myshrimp', 15, 1, 3, 1, 0, 'shrimp')
aqua.add_animal('myscalar', 17, 31, 10, 1, 0, 'scalar')
aqua.add_animal('mymolly', 13, 1, 3, 0, 0, 'molly')
print(aqua)
print(repr(aqua))
aqua.several_steps(8)
print(aqua)
print(repr(aqua))
aqua.several_steps(10)
```

```
print(aqua)
print(repr(aqua))
aqua.several_steps(10)
print(aqua)
print(repr(aqua))
aqua.several_steps(100)
print(aqua)
print(repr(aqua))
```

את ההדפסות של הדוגמא ניתן לראות בקובץ הטקסט "several_steps_example.txt"

:main

לרשותכם קובץ הרצה בשם main. כאשר מריצים את התכנית תחילה מודפס:

Welcome to "The OOP Aquarium"

לאחר מכן מבקשים מהמשתמש לבחור את רוחב האקווריום וגובה האקווריום.

במידה והכניס ערכים תקינים מודפס התפריט הבא:

```
Main menu
-----------------------------
1. Add an animal
2. Print all animals
3. Drop food into the aquarium
4. Take a step forward
5. Take several steps
6. Exit
What do you want to do?
```

אפשרות 1:

כאשר נבחרת אפשרות 1 מודפס למשתמש התפריט הבא:

```
Please select:
1. Scalar
2. Molly
3. Ocypode
4. Shrimp
What animal do you want to put in the aquarium?
```

לאחר בחירת אפשרות תקינה התוכנה תבקש מהמשתמש להזין את פרטי החיה שבחר. במידה והמיקום שנבחר פנוי, החיה תוסף לאקווריום.

לאחר מכן יודפס האקווריום לפי שיטת repr של המחלקה.

אפשרות 2: מדפיסה את האקווריום לפי שיטת ה-str של המחלקה.

אפשרות 3: מאכילה את כל חיות האקווריום ב-10 יחידות אוכל.

אפשרות 4: תבצע צעד אחד באקווריום. לאחר מכן יודפס האקווריום לפי שיטת repr של המחלקה.

אפשרות 5: יבוצעו כמה צעדים באקווריום לפי בחירת המשתמש. לאחר מכן יודפס האקווריום לפי שיטת repr של המחלקה.

אפשרות 6: תסיים את ריצת התוכנה.

קובץ זה מומש עבורכם ובמידה ומימשתם את המחלקות כנדרש הוא יעבוד ללא בעיה.

דוגמאות עבור ריצת קובץ זה ניתן למצוא בקבצים:

main_example.txt

ניתן ורצוי לנסות להריץ באותו האופן את ה-main עם המחלקות שמימשתם ולהשתמש במשוואה טקסט אינטרנטי על מנת לבדוק את ההדפסות של עבודתכם.

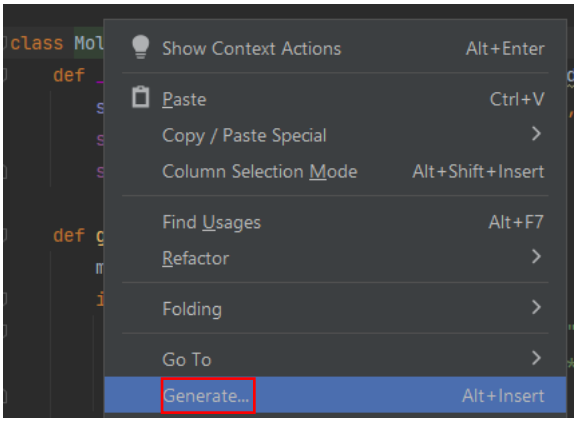
חלק ב' - Unit test

זאת הפעם הראשונה שהינכם מתמודדים עם מספר רב של קבצים, שיטות ומחלקות. על מנת שתוכלו לבדוק את התרגיל שלכם בצורה קלה ונוחה תשתמשו בכלי בדיקות תכנה שנקרא **unit test**, שישמש אתכם בהמשך דרככם המקצועית.

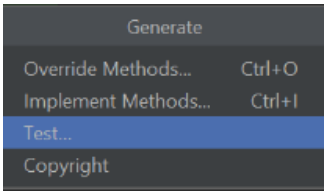
שלבים לפתיחת קובץ Test

על מנת לפתוח קובץ test בצורה נוחה בצעו את השלבים הבאים:

לחצן ימיני בעכבר על שם המחלקה ובחירה ב-Generate.



יפתח החלון הבא:

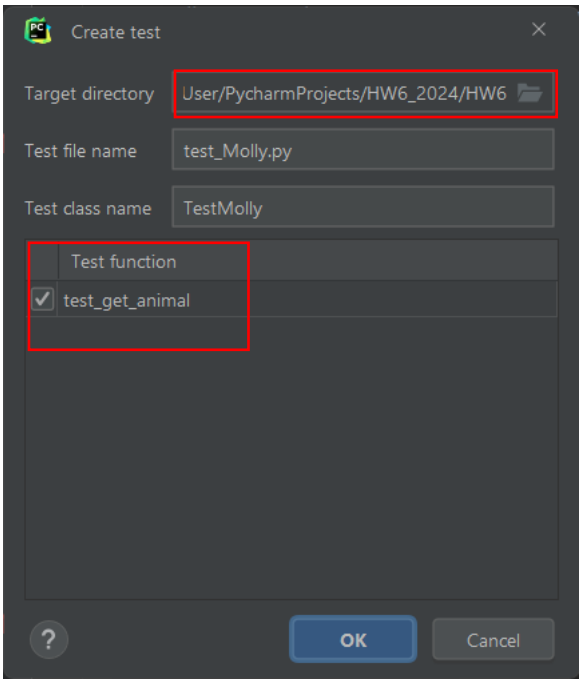


יש לבחור ב-...Test

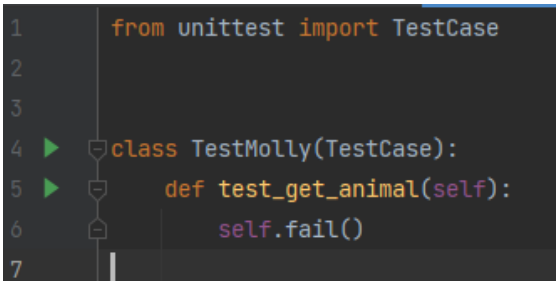
בתיקיית היעד (target directory) הגדירו את התיקייה בה כתבתם את קבצי התרגיל.

שם הקובץ ושם המחלקה נתון לבחירתכם.

סמנו ב- V את כל השיטות המופיעות תחת קטגוריות test function על מנת ליצור טסטים מובנים.



לאחר לחיצה על OK יוצר הקובץ הבא:



שימו לב כי לא נוצרו טסטים אוטומטים לשיטות שדרסתם (override) כמו __str__ או לשיטות שהמחלקה ירשה ממחלקת אב ועליכם ליצר להם טסטים בצורה ידנית.

בנוסף, על מנת לקרוא למחלקה אותה אנו רוצים לבדוק, יש לייבא את הקובץ שלה על ידי הפקודה:

from <fileName> import <className>

שלבים להגדרת test חדש

כדי ליצור טסט חדש נגדיר פונקציה כך ששם הפונקציה מתחיל עם המילה test. למשל:

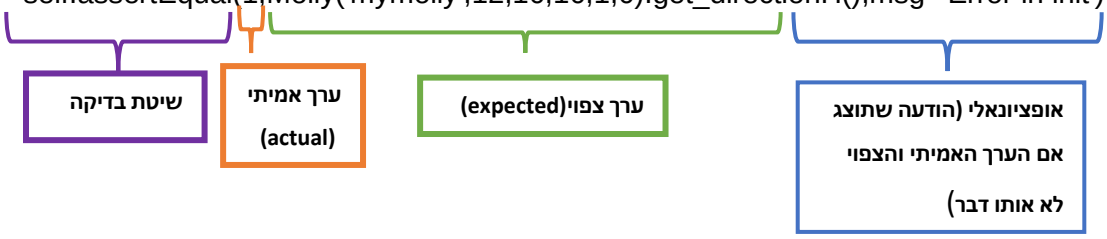
```
def test_str(self):
```

בתוך הפונקציה נבצע תרחיש מסוים שבודק שיטה בקוד. למשל, בדוגמה אנו בודקים כי הבנאי של המחלקה Customer מבוצע כהלכה.

על מנת להריץ את ה-test, עלינו להשתמש באובייקט self ובשיטת בדיקה.

למשל, בדוגמה:

```
self.assertEqual(1,Molly('mymolly',12,10,10,1,0).get_directionH(),msg='Error in init')
```



מוזמנים לקרוא על כל השיטות להשוואה בקישור.

ניתן להריץ טסט בודד על ידי לחיצה על החץ הירוק בצד שמאל ליד שם הפונקציה:

```
class TestMolly(TestCase):
    def setUp(self):
        self.molly = Molly('mymolly',12,10,10,1,0)
    def test_init(self):
        self.assertEqual(1,self.molly.get_directionH(),msg='Error in init')
```

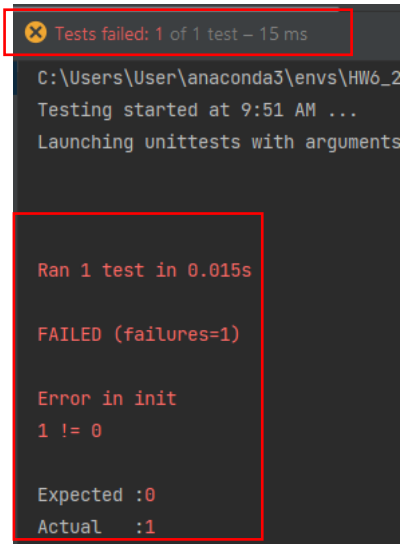
לאחר ההרצה נקבל פלט כי הטסט עבר:

```
Tests passed: 1 of 1 test – 15 ms
C:\Users\User\anaconda3\envs\HW6_2024\python
Testing started at 6:54 PM ...
Launching unittests with arguments python -m


Ran 1 test in 0.002s

OK

Process finished with exit code 0
```

אם למשל נשנה את הטסט לטסט שגוי:

```
class TestMolly(TestCase):
    def setUp(self):
        self.molly = Molly('mymolly',12,10,10,1,0)
    def test_init(self):
        self.assertEqual(0,self.molly.get_directionH(),msg='Error in init')
```

כאשר נריץ נקבל הודעה כי הטסט נכשל:



על מנת להריץ את כל הטסטים לחצו על החץ הירוק ליד שם המחלקה:

```
class TestMolly(TestCase):
    def setUp(self):
        self.molly = Molly('mymolly',12,10,10,1,0)
    def test_init(self):
        self.assertEqual(0,self.molly.get_directionH(),msg='Error in init')
```

דרישות התרגיל

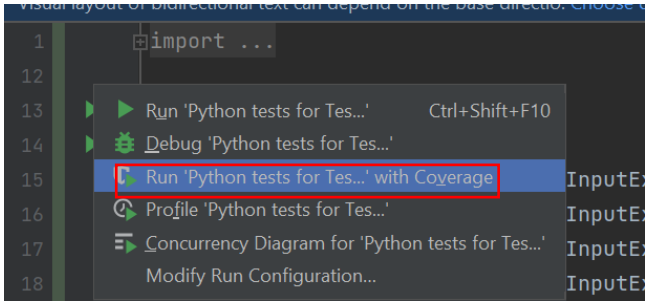
בחלק מדרישות העבודה עליכם לכתוב קובץ test אחד עבור מחלקת **Molly**.

חישבו: כיצד אתם יכולים לכסות את כל מקרי הקצה שיש בשיטה אותה אתם בודקים.

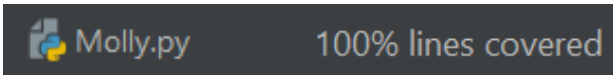
המלצה חמה: לכל מחלקה נסו לכתוב טסטים לפני שאתם כותבים את הקוד על פי השיטה של **TDD** (test driven development) שהוצגה בתרגול 7. באופן הזה תממשו את הקוד כדי שיעבור את הטסטים ולא להיפך. זוהי שיטה מקובלת לפיתוח תכנה.

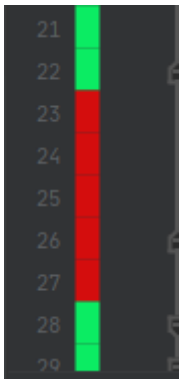
המלצה חמה: דרישות התרגיל הינם לממש unit test עבור מחלקה אחת בלבד. אנו ממליצים לכם לממש עבור כל המחלקות שיצרתם.

כדי שתוכל לדעת האם הטסטים שלכם מכסים את מרבית הקוד שכתבתם תוכלו להשתמש באופציית ה-coverage. לחצן ימני על החץ הירוק ובחירה "run with coverage".



לאחר ההרצה תוכל לראות את השורות שהטסטים כיסו ליד שם המחלקה. למשל:





בנוסף, כאשר תיכנסו לקובץ של המחלקה תוכלו לראות את הצבעים הבאים:

- ירוק – הטסט עבר בשורה זאת ולכן השורה נבדקה.
- אדום – הטסט לא עבר בשורה זאת ולכן השורה לא נבדקה.

לקריאה נוספת על ה-coverage מוזמנים לקרוא כאן .

שימו לב כי בדיקת ה-coverage אינה חובה והיא בשבילכם בלבד.

שימו לב כי אסור לשתף טסטים בין סטודנטים.

בהצלחה ועבודה מהנה!